

HITL Systems — When AI Shouldn't Decide Alone

Week 4 — Turing x UCSB x ACM

Irene Fan & Agnibha Sarkar | April 3, 2026

irenefanyiran@gmail.com · agnibha.sarkar@turing.com

Schedule

Week	Topic
1	Course Kickoff + Problem Scoping
2	Agentic AI & Fundamentals
3	LangGraph & RAG
4	HITL Systems
5	[Guest Lecturer] AI Prototyping / Why Most AI POCs Fail
6	Architecture Design & Eng Design Doc
7	[Student Presentation] Architecture Presentations + Critique
8	Coding Copilots in Practice & Dev Cycle Integration
9	End-to-End Implementation & Evaluation
10	[Student Presentation] Demo Week

Today

Part A — Irene (~15 min)

- 1. HITL: What & Why
- 2. The Biggest Problems in HITL Design
- 3. The 3 Types of HITL
- 4. Real HITL Examples in the Wild

Part B — Agnibha (~35 min)

- 5. Quick Recap: Week 3
 - 6. The Full CBRE Pipeline
 - 7. Applying the 3 Types to CBRE
 - 8. HITL in LangGraph — Interrupt, Review, Resume
 - 9. Live Demo
 - 10. In-Class Exercise: Design the HITL Experience
 - 11. Quiz, Food for Thought, Assignment + Reading
-

Part A: HITL Overview

Irene

HITL. What Is It & Why Do We Need It?

AI agents are getting powerful — they can browse the web, write code, classify problems, dispatch teams.

But **power without oversight is dangerous**.

An AI that's right 95% of the time still gets it wrong 1 in 20. At CBRE's scale (thousands of calls/day), that's dozens of wrong dispatches, wrong severity calls, wrong 911 calls — **every day**.

HITL = designing the system so humans are involved **at the right moments**, not everywhere and not nowhere.

It's Not Just Best Practice — It's the Law

The **EU AI Act (Article 14)** requires that high-risk AI systems "be designed and developed in such a way that they can be effectively overseen by natural persons during the period in which they are in use." This includes manual intervention, overriding, and real-time monitoring. The humans involved must understand the system's capabilities and limitations, and have authority to intervene.

CBRE's dispatch system — routing emergency services, affecting life safety — would likely qualify as high-risk. HITL isn't optional here.

What's the Biggest Problem?

"AI is wrong all the time, so I stopped using it."

This is the failure mode. Not that the AI is bad — but that **the system around it doesn't help humans catch and correct mistakes**.

When users lose trust in an AI system, they stop using it entirely. HITL isn't just about safety — it's about **keeping humans engaged** so the system gets used at all.

Top HITL Problems in Agentic Architecture

Three recurring failures:

1. Poor information surfacing strategy The AI makes a decision, but the human can't see *why*. No reasoning, no confidence score, no alternative options. The human is asked to approve something they can't evaluate.

2. No early correction loop Users can't easily steer or fix direction. By the time they see the output, the AI has already taken 10 steps in the wrong direction. Correction is expensive.

3. No learning from mistakes Same errors repeat. Human corrections go into a void — the system doesn't get better over time. Dispatchers fix the same misclassification every week.

More Problems in Production-Level HITL Design

These show up when you actually deploy:

- **Poorly designed review thresholds** — "low confidence" ≠ "high risk." Risk = Probability of error × Cost of that error. A low-confidence classification of a soap dispenser refill is not the same as a low-confidence gas leak.
- **Humans are not ground truth** — Dispatchers make mistakes too. They're tired, they misread transcripts, they have biases. Training on wrong corrections makes the system worse.
- **HITL degrades over time** — Human attention drifts (approval fatigue). The model drifts (data distribution changes). Neither is monitored.
- **No rollout and change management plan** — You can't drop HITL into an existing workflow without changing the workflow. Dispatchers aren't reviewing AI suggestions today — that's a new job.
- **Not restructuring roles along with HITL** — If dispatchers now spend 40% of their time reviewing AI decisions instead of answering calls, you need to rethink staffing, not just add a dashboard.

The Honest Tradeoffs of HITL

HITL isn't free. IBM identifies three core tensions:

Benefit	Tradeoff
Accuracy & reliability — humans catch edge cases, fix bias, verify high-stakes decisions	Scalability & cost — human review is slow and expensive. Labeling, reviewing, approving at scale = bottleneck
Ethical decision-making & accountability — humans can reason about context, norms, grey areas. Audit trail for compliance	Human error & inconsistency — humans get tired, distracted, biased. Different reviewers may label the same case differently
Transparency & explainability — human oversight mitigates the "black box" effect	Privacy & security — humans in the loop see sensitive data. Well-intentioned reviewers can still leak or misuse information

The goal isn't "add humans everywhere." It's **add humans where the cost of AI error exceeds the cost of human review.**

The 3 Types of HITL

Not all human involvement is the same. There are three fundamentally different roles:

Human Role	When It Acts	What It Affects
------------	--------------	-----------------

Human Role	When It Acts	What It Affects
Trainer	After calls (offline)	Future call handling & routing
Override	During call or workflow	Call trajectory & real-time decisions
Validator	Before action is executed	Current ticket / work order outcome

Each maps to a different design pattern, a different UX, and different LangGraph primitives.

Type 1: Trainer

The human teaches the AI — after the fact.

The system makes a decision. A human reviews it (now or later) and provides a correction. That correction gets logged and used to improve the system — better prompts, better training data, fine-tuning.

When: Asynchronous — doesn't block the pipeline.

Pattern: Feedback loop. Log predictions + corrections. Periodically analyze and improve.

Type 2: Validator

The human approves before the action happens.

The system proposes an action. A human reviews it and says "yes, proceed" or "no, change this." Nothing happens until the human confirms.

When: Before the action — blocks the pipeline until approved.

Pattern: Pre-execution gate.

Type 3: Override Agent

The human corrects after the AI has already acted.

The system makes a decision and executes it. A human can review the outcome and override if needed. The override takes effect and the original action is reversed or corrected.

When: After the action — the system already acted, human corrects.

Pattern: Post-execution correction.

Most Teams Start With Override

It's the easiest to build — the AI acts, humans fix mistakes later.

But think about it:

- **Override** means the wrong action already happened. 911 was already called.

- **Validator** prevents the wrong action from happening.
- **Trainer** prevents the wrong action from ever being proposed.

Override handles today's mistakes. Validator prevents tomorrow's. Trainer eliminates next month's.

The ideal system has all three.

Real HITL Design Examples in the Wild

Magentic-UI (Microsoft Research)

github.com/microsoft/magentic-ui — open-source, MIT licensed, 9.8k stars.

A **human-centered web agent** that implements all 3 HITL types:

Feature	HITL Type	What It Does
Co-Planning	Validator	Agent creates a plan. Human reviews, edits, approves BEFORE execution.
Action Guards	Validator	Agent pauses before irreversible actions. Human approves or denies.
Co-Tasking	Override	Human pauses, takes control of the browser, corrects, hands back.
Plan Learning	Trainer	System learns from completed interactions. Saves plans for reuse.

Key result: even lightweight human feedback (intervening in only 10% of tasks) improved task completion by **71%**.

Claude Code: Plan Mode & AskMeQuestions

Anthropic's coding agent has two HITL patterns baked in:

- **Plan Mode** = Validator. Before writing any code, Claude creates a plan and waits for your approval. You can edit the plan, reject it, or approve. Nothing executes until you say go.
- **AskMeQuestions tool** = the AI decides it needs human input. Instead of guessing, it explicitly asks. This is a validator that the AI triggers itself — "I'm not sure, let me ask."

The pattern: the agent knows its own uncertainty and **requests help rather than guessing wrong**.

Cursor: Review / Accept / Reject

You're using this right now.

- Cursor proposes code changes. You **review** the diff, then **accept** or **reject**. That's a Validator — pre-action gate on every edit.

- When you reject and re-prompt with corrections, that's Override + Trainer — you're fixing the immediate output AND teaching the model what you want.
- The "apply" step is a deliberate HITL checkpoint — the AI never writes to your file without your approval.

These aren't theoretical patterns. **You interact with HITL systems every time you use an AI coding tool.**

Part B: Applying HITL to CBRE

Agnibha

Quick Recap: Week 3

Last week:

- **RAG** — why retrieval beats stuffing everything in the prompt
- **Embeddings + vector stores** — turning meaning into searchable math
- **LangGraph** — nodes, edges, state, wiring a pipeline
- **Demo** — extract → retrieve → classify on CBRE transcripts
- **Assignment** — build agentic RAG with document grading and query rewriting

We built steps 1-3 of the pipeline. Now let's apply Irene's HITL framework to what comes next.

Let's Revisit: Food for Thought from Week 3

FT-3 (from last week)

Our pipeline is linear: extract → retrieve → classify. Real maintenance calls are messy. **What would a non-linear version look like?**

Today's answer: **branches for human review.**

FT-5 (from last week)

We're building an automated call routing system. The previous CBRE system was calling 911 for leaky faucets. **How is what we're building fundamentally different?**

The Full CBRE Pipeline

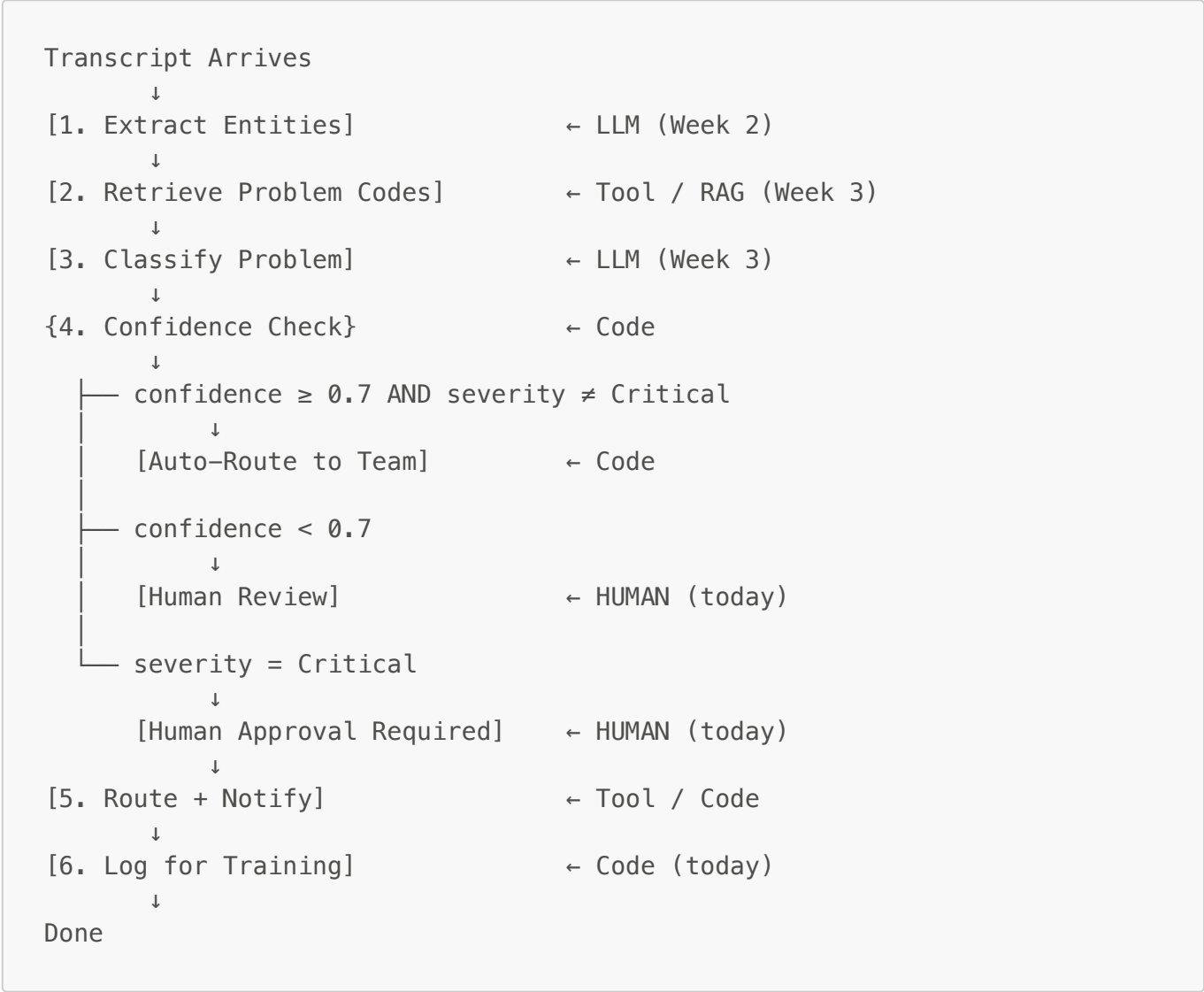
We've been building this in pieces. Time to see the whole thing.

What We've Built So Far

Week	What We Built	Pipeline Step
2	Entity extraction	Transcript → Entities
3	RAG + Classification	Entities → Retrieve codes → Classify
4	HITL + Routing (today)	Classify → Check → Route (with humans)

Steps 1-3 are done. Steps 4-6 are what makes this a **production system** instead of a demo.

The Complete Pipeline



Who Does What

Step	Who	Why
Extract entities	LLM	Unstructured text → structured data
Retrieve codes	Tool (vector search)	Matching against known codes
Classify problem	LLM	Judgment call using extracted info + retrieved codes

Step	Who	Why
Confidence check	Code	Threshold check — deterministic
Human review / approval	Human	Stakes too high for AI to decide alone
Route + notify	Code / Tool	Business rules + API calls
Log corrections	Code	Capture human decisions for future improvement

This is the first time we're seeing the **full pipeline**. Everything before this was building individual pieces.

Applying the 3 HITL Types to CBRE

Now let's map Irene's framework onto our specific pipeline.

Trainer in CBRE

The dispatcher teaches the AI.

The system classifies a call as PLUMB-001 (Pipe Leak). The dispatcher knows it's actually PLUMB-002 (Roof Leak) because the caller mentioned rain. The dispatcher's correction is logged. Over time, the extraction prompt is updated to pay more attention to weather context.

Pattern: Log predictions + human corrections. Periodically analyze and improve prompts or fine-tune.

In LangGraph: A `log_result` node that captures every decision — auto-routed, approved, and overridden.

Validator in CBRE

The dispatcher approves before routing.

The system classifies a call as Critical severity and wants to dispatch 911. Before the call goes out, a human dispatcher sees: "Critical — Gas leak at Westfield Tower, 3rd floor. Dispatch 911?" They confirm or downgrade.

Triggers: severity = Critical OR confidence < 0.7.

In LangGraph: `interrupt()` pauses the pipeline. `Command(resume=...)` sends the human's decision back.

This is the **active learning** pattern from ML: the model identifies its own uncertain or low-confidence predictions and requests human input only where needed. This concentrates human effort on the hardest, most ambiguous cases — not every call.

Remember Irene's point: "low confidence" ≠ "high risk." Risk = Probability of error × Cost of error. A low-confidence soap dispenser call isn't the same as a low-confidence gas leak.

Override in CBRE

The supervisor corrects after routing.

The system auto-routed a broken door call to general maintenance. A supervisor reviews the queue, sees it's actually a fire door (safety-critical), and overrides the routing to the priority fire safety team.

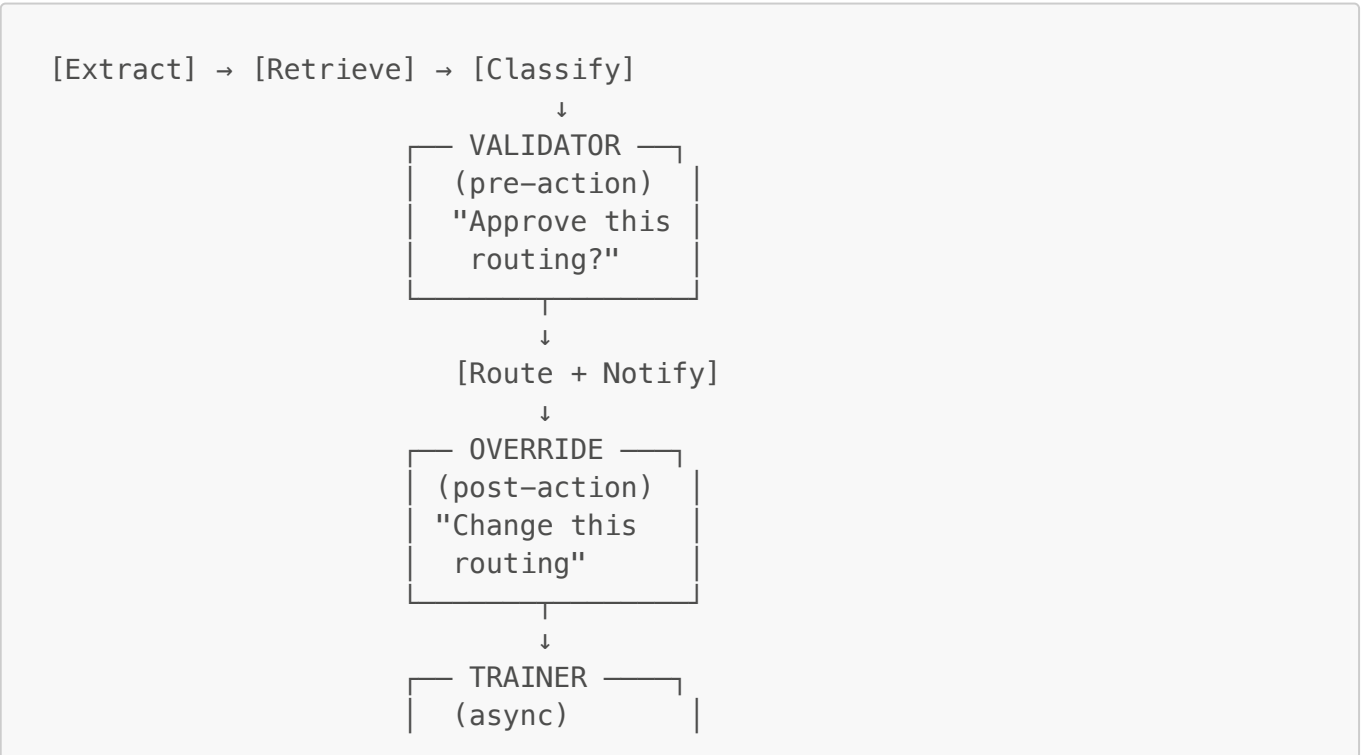
Pattern: Post-execution correction. The wrong team already got dispatched — we're fixing it after the fact.

In LangGraph: `update_state()` changes the classification in the saved checkpoint. Downstream systems see the corrected version.

Comparing the 3 Types (CBRE-Specific)

	Trainer	Validator	Override
When	After (async)	Before action	After action
Blocks pipeline?	No	Yes	No
Impact	Future improvement	Prevents bad actions	Corrects bad actions
Cost	Low (batch review)	High (every critical call)	Medium (spot-check)
CBRE use	Log corrections for prompt/model improvement	Approve Critical severity routing	Supervisor overrides wrong routing
LangGraph	<code>log_result</code> node	<code>interrupt()</code> + <code>Command(resume=...)</code>	<code>update_state()</code>

Where Do They Go in the Pipeline?



```
"Log correction  
for learning"
```

HITL in LangGraph

LangGraph Makes HITL Explicit

In plain Python, HITL is messy — `input()` calls buried in functions, global flags, no way to pause and resume.

LangGraph treats HITL as a first-class concept:

- **Checkpoint** — saves the full pipeline state so you can pause and resume
- `interrupt()` — pauses execution and asks for human input
- `update_state()` — lets a human modify the pipeline state
- `interrupt_before` / `interrupt_after` — pause at specific nodes

The Checkpointer: Enabling Pause + Resume

```
from langgraph.checkpoint.memory import InMemorySaver

memory = InMemorySaver()
pipeline = workflow.compile(checkpointer=memory)
```

Without a checkpointer, the pipeline runs start-to-finish with no way to stop. The checkpointer saves state at every node so you can pause, inspect, modify, and resume.

`interrupt()` — Pausing for Human Input

```
from langgraph.types import interrupt

def validate_routing(state):
    classification = state["classification"]

    if classification["severity"] == "Critical" or  
classification["confidence"] < 0.7:
        decision = interrupt({
            "question": "Approve this routing?",
            "classification": classification,
        })

        if not decision.get("approved"):
            return {"classification": decision["revised"]}
```

```
return state
```

When `interrupt()` is called, the pipeline **pauses**. It returns control to the caller. When you invoke the pipeline again with the same thread, it **resumes** from where it stopped.

update_state() — Human Override

```
config = {"configurable": {"thread_id": "call-001"}}

result = pipeline.invoke({"transcript": transcript}, config)

# Supervisor reviews and wants to change the routing
pipeline.update_state(
    config,
    {"classification": {"selected_code": "SAFE-002", "confidence": 1.0,
                       "reasoning": "Supervisor override: fire door, not
general door"}},
)
```

This changes the pipeline's saved state — any downstream node that reads classification will see the human's version.

The Flow: Invoke → Pause → Review → Resume

```
from langgraph.types import Command

config = {"configurable": {"thread_id": "call-001"}}

# Step 1: Run until interrupt
result = pipeline.invoke({"transcript": transcript}, config)

# Step 2: Check what happened
state = pipeline.get_state(config)
print(f"Paused before: {state.next}")
print(f"Classification: {state.values['classification']}")

# Step 3: Human decides — pass decision back via Command(resume=...)
# The value becomes the return value of interrupt() inside the node.

# Approve:
result = pipeline.invoke(Command(resume={"approved": True}), config)

# Or override:
result = pipeline.invoke(
    Command(resume={"override_code": "SAFE-002", "reason": "Fire door"}),
```

```
    config,  
)
```

Demo Time

[hitl_demo.ipynb](#)

The Week 3 pipeline + HITL checkpoints:

1. Extract → Retrieve → Classify (same as Week 3)
2. **Validator gate** — pauses on Critical severity or low confidence
3. **Override** — change the classification after routing
4. **Trainer log** — capture human corrections

In-Class Exercise: Design the HITL Experience

Your Turn

5-8 minutes in pairs or small groups.

Look at the full CBRE pipeline diagram. For each of the 3 HITL types:

1. **WHERE** in the pipeline would you put it? (which node, which edge)
2. **WHEN** should it trigger? (what condition — always? only for certain severities? only when confidence is low?)
3. **WHAT** does the human see? (what information do they need to make the decision?)

Be specific. Think about tradeoffs — every human checkpoint adds latency.

Think about what Irene said: Risk = Probability of error × Cost of error.

Discussion: Share Your Designs

- Which decisions did you let the AI handle alone?
- Where did you add human gates?
- What tradeoffs did you make between speed and safety?
- Did any group add a human checkpoint that another group left out?

Quiz: Check Your Understanding

Q1

Your CBRE pipeline auto-routes Low and Medium severity calls but requires human approval for High and Critical. A product manager says: "High severity calls are 30% of volume. Adding human review for all of

them will create a bottleneck — average response time goes from 10 seconds to 4 minutes." What's the best response?

- (A) Remove human review for High — only keep it for Critical
 - (B) Keep human review for all High and Critical — safety is more important than speed
 - (C) Add a secondary filter: only interrupt High severity calls when confidence is also below 0.8. High-confidence High-severity calls can auto-route; low-confidence ones get human review
 - (D) Replace human review with a reflection step — an LLM reviewing another LLM is faster
-

Q2

Your system classifies a call as HVAC-001 (AC Failure) with 0.92 confidence. It auto-routes to the HVAC team. Two hours later, the HVAC tech reports it's actually an electrical issue — the AC failed because of a power outage on that floor. Which HITL type would have caught this, and which is the right response now?

- (A) Validator — the system should have paused for approval since HVAC issues sometimes have electrical root causes
 - (B) Override — the supervisor should reclassify to ELEC-001 and reroute. Then log the correction as training data so the system learns that AC failure + other equipment down = check electrical first
 - (C) Trainer — this is a rare edge case that doesn't justify adding a validator gate for every HVAC call
 - (D) Both B and C — override to fix the immediate problem, then use the correction as training signal. Adding a validator for every HVAC call would create unnecessary bottlenecks
-

Q3

You're implementing a validator gate in LangGraph. You compile the graph with `interrupt_before=["route"]`. A teammate asks: "Why not just add an `if` statement inside the route node that calls `input()`?" What's the advantage of using LangGraph's interrupt system?

- (A) There's no real advantage — `input()` does the same thing
 - (B) LangGraph's interrupt saves the full pipeline state via the checkpoint. The human can review hours later, on a different machine, and the pipeline resumes exactly where it left off. An `input()` call blocks the Python process and loses all state if it crashes
 - (C) LangGraph interrupts are faster than `input()`
 - (D) `input()` can't be used in Jupyter notebooks
-

Q4

Your pipeline has a trainer loop: every human correction is logged with the original prediction and the corrected label. After 3 months, you have 500 corrections. A teammate says: "Great, let's fine-tune the model on these." What's the risk?

- (A) 500 examples isn't enough to fine-tune any model
- (B) The corrections are biased toward the cases the system got wrong — fine-tuning only on failures could make the model worse on cases it was already handling correctly. You need to mix corrections with examples of correct predictions
- (C) Fine-tuning is always better than prompt engineering

- (D) The corrections are too old to be useful after 3 months
-

Q5

Your validator gate interrupts on all Critical severity calls. A gas leak call comes in at 2 AM. The system classifies it as Critical with 0.98 confidence and pauses for human approval. The on-call dispatcher is asleep and doesn't respond for 12 minutes. What does this reveal about your HITL design?

- (A) The validator gate is working correctly — Critical calls should always wait for human approval
 - (B) The design has a latency-safety tradeoff problem. For genuinely high-confidence life-safety situations, the system needs a timeout with auto-escalation — if no human responds within N minutes, auto-route AND alert a backup. HITL shouldn't create a worse outcome than no HITL
 - (C) Remove the validator gate for Critical calls — they should always auto-route
 - (D) Add more on-call dispatchers to ensure 24/7 coverage
-

Q6

You implement all 3 HITL types. After a month, you notice: validators approve 98% of calls without changes, overrides happen on 1% of calls, and the trainer log has collected 150 corrections. A teammate says: "The validator is a waste — humans approve almost everything." How do you evaluate this?

- (A) They're right — remove the validator to reduce latency
 - (B) 98% approval rate doesn't mean the validator is useless. The 2% it catches could be the high-stakes cases (false Critical, misrouted gas leaks) where preventing one mistake justifies reviewing 50 correct ones. Evaluate the *impact* of the 2%, not just the percentage
 - (C) The validator is definitely necessary — any human review step is valuable
 - (D) Switch the validator to only trigger on low-confidence calls to reduce the 98% pass-through
-

Q7

A junior engineer implements the override pattern: after the system routes a call, supervisors can reclassify and reroute. But they don't log the override — they just change the routing. Why is this a problem?

- (A) It's not a problem — the call gets to the right team, which is what matters
 - (B) Without logging, the override is only fixing today's mistake. The system will make the same mistake on the next similar call. Logging the override (original prediction + human correction + reasoning) converts an Override into a Trainer signal — it fixes the immediate problem AND prevents future ones
 - (C) Logging creates privacy concerns
 - (D) Overrides should never be logged because they indicate system failure
-

Q8

You're deciding where to put a validator gate. Option A: after classification, before routing (interrupt if severity is Critical). Option B: after routing, before notification (interrupt if the notification involves 911 or emergency services). Which is better, and why?

- (A) Option A — catch problems earlier before any action is taken
 - (B) Option B — more targeted. Not every Critical classification leads to 911. By gating at the notification step, you only interrupt the truly irreversible actions (emergency dispatch) and let non-emergency Critical routing proceed automatically
 - (C) Both — add gates at both points for maximum safety
 - (D) Neither — use reflection instead of human review
-

Q9

Your trainer log reveals a pattern: 40% of overrides are cases where the system classified a compound problem (e.g., water leak near electrical equipment) as a single-category issue. The dispatchers consistently reclassify these as the higher-severity category. What's the most effective way to use this insight?

- (A) Fine-tune the model on these 40% of corrections
 - (B) Update the extraction prompt to explicitly check for compound hazards, add examples of compound problems to the severity guidelines, and add a reflection step that cross-references problem type with location for safety implications. Then measure if the override rate for compound problems drops
 - (C) Add a validator gate specifically for calls that mention multiple problem types
 - (D) Both B and C — improve the prompt AND add a safety gate for compound problems until the prompt fix is proven
-

Q10

Your team is debating the HITL design for the full CBRE system at scale (10,000 calls/day). Currently, validators review 15% of calls, overrides happen on 2%, and the trainer loop runs weekly. A product manager wants to reduce human involvement to cut costs. Rank these approaches from most to least effective:

- (A) Remove validators entirely and rely only on overrides — cheapest option
 - (B) Use the trainer data to improve the model until validator approval rate hits 99.5%, then remove validators for the high-confidence cases — only keep them for genuinely uncertain calls
 - (C) Replace human validators with an LLM-as-Judge that reviews classifications before routing
 - (D) B is best, then C as a supplement, then A as last resort. The goal is to shrink the validator pool through system improvement (B), augment remaining review with LLM pre-screening (C), and only remove validators as a last resort when the data proves it's safe (not A as a cost-cutting shortcut)
-

Quiz Answers

Q	Answer	Why
1	C	Blanket rules waste human time. A secondary filter (severity + confidence) keeps humans focused on genuinely uncertain cases. 70% of High calls at high confidence can safely auto-route.

Q	Answer	Why
2	D	Override fixes the immediate misroute. Training signal prevents future AC-failure-with-power-outage misclassifications. A validator for every HVAC call is overkill for a rare edge case.
3	B	LangGraph interrupts persist state via the checkpoint. The human can review asynchronously, on a different machine, hours later. <code>input()</code> blocks the process and is fragile.
4	B	Training only on failures creates selection bias. The model needs to see what it's doing right too. Mix corrections with correct predictions for balanced fine-tuning.
5	B	HITL shouldn't make outcomes worse. A validator with no timeout on a life-safety call is a design flaw. Add auto-escalation with a timeout — the human can still override after.
6	B	Evaluate by impact, not volume. If the 2% catches are high-stakes mistakes (false 911, misrouted gas leaks), the validator is worth it. Also consider D as an optimization.
7	B	Unlogged overrides fix one call. Logged overrides fix all future similar calls. Override without logging is a missed Trainer opportunity.
8	B	Gate the irreversible action, not the classification. Critical doesn't always mean 911. Gating at notification is more targeted and creates fewer unnecessary interruptions.
9	D	Improve the system (B) so it stops making compound-hazard mistakes, AND add a safety gate (C) as a backup until the improvement is proven. Belt and suspenders.
10	D	Improve the model using trainer data until validators are rarely needed (B). Use LLM pre-screening to handle the easy reviews (C). Only remove human review when data proves it's safe, never as a cost-cutting shortcut (A is dangerous).

Food for Thought

No right or wrong answers — open discussion prompts.

FT-1

Your validator gate catches a misclassified gas leak and prevents a bad routing decision. But the gas leak was real and the 3-minute delay while waiting for human approval meant the response was slower. **When does the safety benefit of human review get outweighed by the delay it introduces?** Is there a severity level where speed matters more than accuracy?

FT-2

The trainer pattern assumes human corrections are right. But dispatchers make mistakes too — they're tired, they misread the transcript, they have biases. **What happens when you train your AI on incorrect human corrections?** How do you quality-control the quality controller?

FT-3

Your system auto-routes 85% of calls, validators review 10%, and overrides fix 5%. Over 6 months, the trainer loop improves the model and auto-routing rises to 95%. **Should you ever reach 100% automation?** What would have to be true for you to remove humans entirely from a life-safety pipeline?

FT-4

You're building HITL for the CBRE system. The dispatchers who will review AI decisions currently handle calls manually — they answer phones, classify problems, and dispatch teams. **How do you design the validator UI so it helps them rather than slowing them down?** What information should be shown? What shouldn't? How is reviewing an AI suggestion different from making the decision from scratch?

FT-5

LLM-as-Judge can replace some human review — an LLM reviews another LLM's output. This is faster and cheaper than human validators. **But is LLM-as-Judge really "human-in-the-loop," or is it just "model-in-the-loop"?** When is an LLM reviewer acceptable, and when do you genuinely need a human?

Assignment

Time estimate: ~3-4 hours

Part 1: Add HITL to Your Pipeline

Take your agentic RAG system from Week 3 (or the demo notebook) and add:

1. **A validator gate** — use LangGraph's `interrupt()` to pause when severity is Critical or confidence is below 0.7. The human should see the classification, confidence, reasoning, and retrieved codes before approving.
2. **An override mechanism** — after routing, allow a human to reclassify using `update_state()`.
3. **A trainer log** — every human correction (from validator edits or overrides) gets appended to a log with the original prediction, the correction, and the timestamp.

You'll need a checkpoint (`InMemorySaver`) and thread configs.

Part 2: Design Document (short)

Write a half-page design note answering:

1. Where did you put each HITL type and why?
 2. What condition triggers the validator? Why that threshold?
 3. What information does the human see when reviewing? What's missing?
 4. If CBRE processes 10,000 calls/day, how would your design scale? What would break first?
-

What to Bring to Week 5

- Your notebook with HITL checkpoints implemented
- Your design document
- One example where the validator changed the outcome (or where it approved something it shouldn't have)

Reading List

Resource	What & Why
Microsoft Magentic-UI	Real-world HITL agent from Microsoft Research. Implements co-planning (validator), action guards (interrupt), co-tasking (override), and plan learning (trainer). Try it locally or read the code — MIT licensed.
Magentic-UI Blog Post	The "why" behind Magentic-UI — how lightweight human feedback improved task completion by 71%.
LangGraph Interrupts & HITL Guide	Official guide to interrupt(), interrupt_before, update_state. Your implementation reference.
What Is Human-in-the-Loop? (IBM)	IBM's overview — benefits, drawbacks, supervised learning, RLHF, active learning, and the EU AI Act mandate.
Human-in-the-Loop Agent Patterns	Covers approval gates, escalation, graduated autonomy — the patterns behind the 3 HITL types.
The Complete Guide to HITL AI (2026)	Comprehensive overview — training, validation, override, and when to use each.
LangGraph Documentation	Reference for nodes, edges, state, checkpointers.

Next Week

Week 5: Industry Realities of Building AI Products

What changes when you go from demo to production?

Cost, latency, reliability, and the things no tutorial teaches you.
